Beyond Federated Learning for IoT: Efficient Split Learning With Caching and Model Customization

Manisha Chawla, Gagan Raj Gupta, Shreyas Gaddam, and Manas Wadhwa

***Abstract*—Distributed training of Deep Learning (DL) models on resource-constrained devices has gained significant interest. Training data-driven DL models collaboratively using federated learning (FL) and split learning (SL) have become the most popular ways to do this task. We aim to optimize these techniques by reducing device computation during parallel model training and reducing high-communication costs due to the frequent exchange of models, data, and gradients. This article proposes efficient SL (ESL), a novel approach addressing these challenges through three key ideas: 1) a key-value store for caching and sharing intermediate activations across clients, significantly reducing redundant computations and communication during the training phase; 2) customization of state-of-the-art neural networks for SL context; and 3) personalized training allowing clients to learn individual models tailored to their specific data distributions. Existing communication-efficient FL/SL methods trade accuracy to reduce communication. In contrast, ESL achieves significant communication reduction while maintaining high accuracy. Extensive experimentation on real-world federated benchmarks for image classification and 3-D segmentation demonstrates significant improvements over baseline FL techniques: ESL achieves a reduction in computation by 1623× for image classification and 23.9× for 3-D segmentation on resource-constrained devices. Additionally, it reduces communication traffic, during training, between clients and the server by 3.92× for image classification and 1.3× for 3-D segmentation while improving average accuracy by 35% and 31%, respectively. Furthermore, when compared to the baseline SL approaches, ESL reduces communication traffic during training by 100× and improves accuracy by an average of 34.8%.**

***Index Terms*—Communication reduction, federated learning (FL), Internet of Things (IoT), key-value store, personalization, resource-constrained devices, split learning (SL).**

I. Introduction

DEEP learning (DL) models [1] consist of a large number of parameters and need significant memory, computational resources, and energy for the training process. Traditionally, they are trained centrally on a server, using a huge amount of data collected from end devices. Nowadays, Internet of Things (IoT) devices produce valuable data, which could be used to learn high-quality DL models that have applications in healthcare [2], surveillance [3], and industrial safety [4]. However, IoT data is usually owned by individuals or organizations whose preferences and policies do not permit sharing data. Federated learning (FL) [5] offers a solution to this challenge. FL enables devices to collaboratively train DL models with local data. The central server securely combines these models, which were trained locally, to update the global model.

Deploying FL on IoT devices presents substantial challenges due to their resource constraints. Training or finetuning state-of-the-art DL models on these devices demands substantial computational resources, often surpassing their processing capabilities. Additionally, the communication overhead during model aggregation poses a significant bottleneck, especially considering the limited bandwidth and connectivity of IoT devices. To overcome these challenges and unlock the potential of DL within the IoT ecosystem, it is imperative to develop innovative techniques for efficient distributed model training on resource-constrained devices. These techniques should aim to optimize communication and computation while ensuring the effective training of state-of-the-art DL models.

Split learning (SL) [6] is an innovative technique that offloads the resource-intensive part of DL model training to a central server without the need for direct access to client data. This is done by splitting a model into two or three submodels. The client-side submodel is trained on the client device with local training data, while the server-side submodel is trained using the activations (at the cut layer) coming from the clients. The computational burden on the clients is reduced, but, the communication burden increases in direct proportion to the number of training samples. In other words, the more data points each edge device has to process, the more "activations" they need to communicate with the server for the training to be effective. Thus, efficient implementation of SL on decentralized IoT devices with realistic benchmarks is an important and active area of research.

Prior studies [7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [17], [18], [19], [20], [21], [22], [23], [24], [25], [26], [27] have achieved efficient implementation of SL/FL by employing established techniques, including gradient sparsification, asynchronous training, and rapid model convergence. However, these may lead to a decrease in the model's accuracy. Our proposed approach, efficient SL (ESL), in contrast to

Manuscript received 12 February 2024; revised 27 April 2024; accepted 25 June 2024. Date of publication 5 July 2024; date of current version 9 October 2024. *(Corresponding author: Manisha Chawla.)*

Manisha Chawla and Gagan Raj Gupta are with the Department of Computer Science and Engineering, Indian Institute of Technology Bhilai, Bhilai 491001, India (e-mail: manishach@iitbhilai.ac.in; gagan@iitbhilai.ac.in).

Shreyas Gaddam is with the Department of Computer Science and Engineering, Shri Shankaracharya Institute of Professional Management and Technology (Raipur), Raipur 492015, India (e-mail: iamshreydan@gmail.com).

Manas Wadhwa is with the Department of Computer Science, University of Massachusets, Amherst, MA 01003 USA (e-mail: mwadhwa@umass.edu).

Digital Object Identifier 10.1109/JIOT.2024.3424660

prior approaches, reduces communication overhead without compromising performance. Additionally, previous research largely neglected to quantify the computation overhead, which we have investigated in depth.

In this article, we introduce ESL to minimize computation and communication overhead in SL to make it more practical and efficient for IoT devices. The proposed design employs a U-shaped architecture to split a DL model into three parts. This split allows us to hide the client's data and labels from the offloading server. ESL employs *transfer learning* [28] with pretrained state-of-the-art models to extract features and speed up convergence. In batch training, where samples are allocated to batches dynamically, the transfer of activations from the client to the server for each batch leads to significant communication overhead. To address this issue, our proposed solution utilizes a novel *server-side key-value store*. This stores *caches* activation values and unique keys associated with each input. Integrating this key-value storage eliminates the need for clients to send activations repeatedly. Instead, the client sends the keys and activations in the first epoch; hence, in subsequent epochs, it only sends distinct keys linked to each sample to the server. This strategy greatly minimizes the required communication by limiting the data transmitted from the client to the server, improving efficiency in overall training.

In SL, different clients' models are averaged during training to create a global generalized model that can perform well on all clients' data distributions. In our proposed design, clients are also allowed to further *personalize their models*[1] to meet their specific requirements, allowing for optimization based on their non-IID data. Furthermore, to achieve optimal performance in SL, it is necessary to determine where to split the model. After evaluating several client-server model splits, we observed that increasing the number of layers on the client side improves performance after personalization for the image classification task. This approach needs additional client-side computations. We propose a novel approach of *integrating a customized block* at the client's back side of the model that provides optimal performance and allows us to offload the maximum layers of the model to the server.

Furthermore, 3-D segmentation models like U-NET [30] have a symmetric architecture with two main parts: 1) the contracting path, which utilizes standard convolutional layers and 2) the expansion path, which is transposed 2-D convolutional layers. This results in larger size activations toward the last few layers of the model. Deciding where to split such a model is quite challenging. Splitting in the middle layers significantly boosts client-side computations and Dice scores, while thinner splits increase communication overhead and result in lower Dice scores. Moreover, the communication overhead increases when each client possesses numerous data points. Thus, the optimal split depends on the available resources. Additionally, we demonstrate a noteworthy reduction in communication and computation by introducing a client-side key-value store within the FL framework.

The key contributions of this article are as follows.

1) Adaptation of transfer learning and personalization in SL framework.
2) A new key-value store design in FL and SL for caching to reduce communication and computation overhead, enhancing efficiency.
3) Optimized performance and resource consumption using additional custom layers at the client's backend to efficiently handle non-IID data distributions.
4) Thorough experimental evaluation using ESL on data sets like CIFAR-10, fashion-MNIST (FMNIST), diabetic retinopathy, real-world medical data [31], and 3-D segmentation scenarios.
5) Our approach achieves a 100× reduction in communication traffic during training compared to baseline SL, maintaining accuracy against centralized training and surpassing baseline FL with transfer learning by an average of 35% and baseline SL by an average of 34.8%. Moreover, our methods show a 1623× reduction in computation time during training on resource-constrained devices compared to baseline FL with transfer learning.

Subsequent sections delve into existing FL/SL methods and their challenges (Section II), the proposed method (Section III), experimental setup (Section IV), results and discussion (Section V), and concluding remarks (Section VI).

II. Related Work

FL and its variants, such as FedAvg [32], FedProx [33], Scaffold [34], and SL [6], solve data privacy concerns in centralized training by limiting data sharing. FL demands extensive computations on client devices, while SL reduces client-side workload by handling only front layers. Split FL (SFL) [35] emerged by allowing parallel model training while minimizing local computations on individual devices. All these techniques face significant challenges in the form of communication overhead due to the exchange of model updates between clients and the central server. To address this challenge, extensive research has been conducted on communication-SFL techniques. We can broadly divide these into two categories.

A. Accelerate Convergence Strategies

One of the primary approaches to communication reduction in FL or SL is to expedite model convergence, thereby reducing the number of communication rounds required. Various methods have been proposed to accelerate convergence, including the following.

1) *Momentum-Based Optimization:* Momentum-based algorithms, such as FedAvg with momentum [32], incorporate a momentum term that helps models converge faster by maintaining a moving average of historical gradients.
2) *Adaptive Learning Rates:* Adaptive learning rate schemes, such as FedLALR [7], adjust the learning rate for each client based on their local data and gradient information, leading to faster and more stable convergence.

[1] we proposed this in PFSL [29].

3) *Asynchronous Training:* Asynchronous training allows the client to update their models independently and communicate with the central server at different times, reducing the need for synchronous communication rounds.

However, it is important to consider potential drawbacks, such as possible degradation in model performance compared to synchronous approaches. Methods, such as VAFL [9] and [24], have explored asynchronous training FL settings. Moreover, Chen et al. [21] explored the use of asynchronous training to reduce communication overhead in SL scenarios.

B. Communication Compression Techniques

Another effective strategy for communication reduction in FSL is to compress model updates directly. This can be achieved through various techniques, including the following:

1) *Model Parameter Compression:* Compressing model updates before transmission can significantly reduce communication overhead. Techniques, such as sparsification [8], quantization [9], and pruning [10], have been employed to compress model parameters.
2) *Gradient Sparsification and Pruning:* Gradient sparsification aims to identify and retain only the most important elements of the gradient, effectively reducing the amount of data transmitted. Methods, such as top-k sparsification [11], [15], have been proposed for gradient sparsification in FL. Zheng et al. [13] proposed a randomized top-k sparsification method to reduce communication overhead in SL (a comparison of this technique with ESL can be found in Section V-F).
3) *Quantization:* Quantization involves reducing the precision of model parameters and gradients, leading to smaller message sizes. Techniques, such as uniform quantization [12], have been explored for quantization in FL. Combining sparsification and quantization [15], [17], [18] will further reduce the communication. Furthermore, Oh et al. [36] has provided communication-ESL via adaptive feature-wise compression.
4) *Singular Value Decomposition (SVD):* SVD can be applied to decompose model updates into lower dimensional representation, reducing the communication overhead. Methods, such as FedSVD [14], PowerSGD [20], and ATOMO [19], utilize SVD for communication compression in FL.
5) *Autoencoder-Based Compression:* Autoencoders can be employed to compress intermediate results during model training, reducing the amount of communication overhead. Methods, such as DAEF [25] and FLAC [26], [27], have utilized autoencoder-based compression for cross-silo FL. Furthermore, Ayad et al. [22] investigates communication and computation efficiency for SL in IoT applications, proposing a novel federated SL architecture with asynchronous training and quantization.

The aforementioned communication compression methods reduce overhead by transmitting fewer bits. It affects the model's accuracy. For instance, in [13], achieving 88% less communication led to a 3% reduction in accuracy. Our research introduces innovative strategies to improve accuracy with reduced communication and computation. Leveraging key-value caching with transfer learning minimizes redundancy. It is a novel approach not explored in previous FL/SL research. Moreover, our approach is compatible with existing communication-efficient strategies like quantization or gradient sparsification. These techniques can be incorporated for further overhead reduction if needed.

Additionally, existing research [37], [38], [39] has focused on optimal splits in SL, yet their impact on performance with IID and non-IID client data remains unexplored. Our study bridges this gap, revealing that optimal splits can increase client-side computational demands. To address this, we customize client-side models for high accuracy with minimal computational overhead—a novel concept in FL/SL, distinguishing our work as impactful. Moreover, we integrate two-phase training, a known FL technique [40], [41], [42], into our SL approach, enhancing communication efficiency and model performance [29].

Furthermore, our methodology is extensively evaluated using real-life federated benchmarks for image classification and 3-D segmentation tasks. Unlike previous studies, we extensively test across six benchmarks, including three from a cross-silo healthcare data set [31], offering genuine hospital splits. This thorough analysis showcases the resilience and applicability of our communication-ESL method, especially in healthcare contexts.

III. System Model

We introduce a novel system model architecture, depicted in Fig. 1(a), designed for ESL. The primary entities in this architecture include clients, which are IoT devices, an offloading server, a key-value store, and a client model aggregation server. This architectural blueprint optimizes computational efficiency by segmenting the pretrained model into three parts: 1) client front; 2) center; and 3) client back model. We illustrate these in Fig. 1(a), using ResNet-18 as an example, demonstrating how a model can be split. In particular, the client is in charge of the first two layers and last two layers. The offloading server is in charge of the remaining intermediate levels. Since the intermediate layers typically involve more computationally demanding operations, the offloading server takes on the majority of these calculations. This setup is called a *thin split* since the client only keeps the very last few layers of the split configuration.

To reduce communication and computation burdens on client devices, we adopt a transfer learning strategy [43]. We utilize pretrained DL models and freeze the front layers' weights at clients, preserving general features like edges and shapes. This aligns with principles from [43], where lower layers learn transferable features. On the server side, we also leverage transfer learning by dividing the pretrained *center model* into a frozen *center-front model* and a trainable *center-back model*. This approach retains pretrained knowledge while allowing adaptation to aggregated client data.

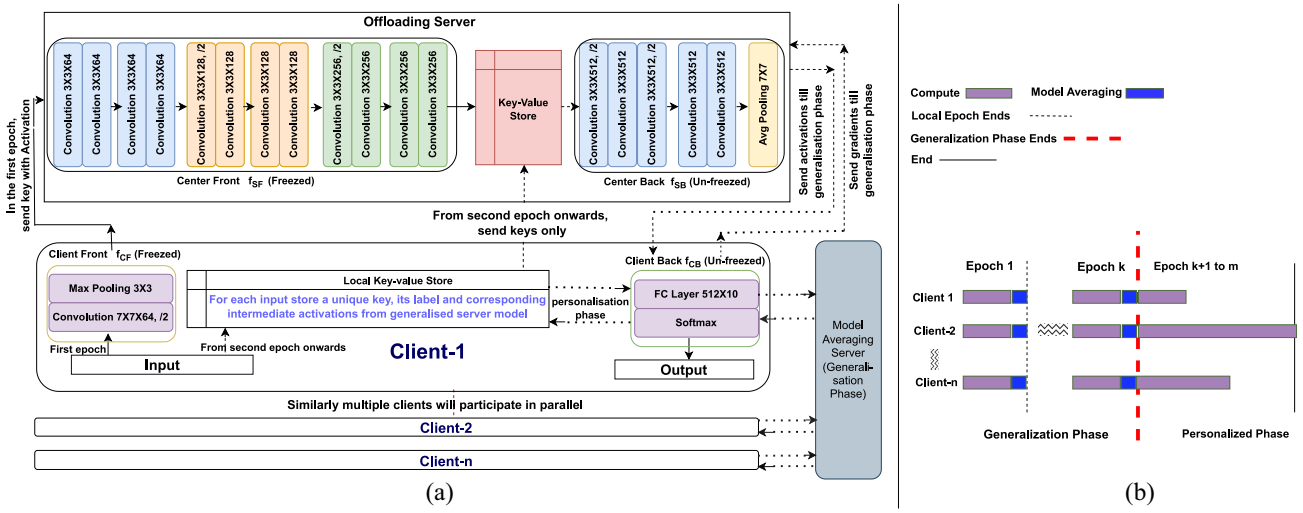

Fig. 1. (a) ESL system architecture using ResNet-18 as an example for splitting the model into the client front, center front, center back, and client back models for training with a key-value store. Solid arrows represent one-time communication during training, while dashed arrows depict communication occurring throughout all epochs. (b) Generalized and personalized training model.

The server operates with limited knowledge, receiving only activations and gradients from the client, thereby safeguarding the client's raw data from potential disclosure. The last layers at the client are lightweight and trainable and primarily serve the purpose of generating output. These layers are more sensitive to the perturbation of their parameters, making them more important for unfreezing and personalization on individual devices [43]. The loss calculation on these client back layers, based on generated outputs, takes place on the client, enabling localized computations without disclosing sensitive label information. By minimizing the computational demands on client devices and allocating heavier computational responsibilities to the server-side models, we have achieved a streamlined and resource-efficient approach. Furthermore, the offloading server is responsible for aggregating all updates made to the intermediary layers by all clients. Meanwhile, the client back layers are updated with the assistance of the model averaging server. Strategically segregating the client model averaging server and offloading server ensures that none of the external entities poses a complete view of the model, thereby enhancing privacy and security.

Within the framework of FL, the general DL model utilized for cooperative training amongst dispersed clients is represented by the function $f(x)$ in (1). Essentially, our approach in SL involves decomposing that function into distinct components that are shared between the client and the server. Functions f_{CF} and f_{SF} correspond to the front layers of the client and server, respectively, while f_{SB} and f_{CB} represent the back layers of the server and client, respectively. We intend to keep f_{CF} and f_{CB} as lightweight as possible to minimize computation and memory requirements for resource-constrained IoT clients. Additionally, we use pretrained weights at the client front (f_{CF}) layers and freeze them. Thus, we do not require any gradients to be passed from the server to the client. We can also freeze parameter updation of the server front part (f_{SF}) to minimize computation load at the server

$$f(x) = \mathbf{f_{CB}}(\mathbf{f_{SB}}(f_{SF}(f_{CF}(x)))). \quad (1)$$

Our architectural design offers extensive flexibility, allowing for the dynamic allocation of layers between client and server. This flexibility empowers us to assess the impact of various splits on task performance. To further reduce the communication overhead and computation and improve performance, we have implemented the following strategies.

A. Two Phase Training

As shown in Fig. 1(b), the training process is comprised of two distinct phases that occur in a sequential order: 1) the first phase is generalization and 2) the second phase is personalization. During the generalization phase, the objective is to create a global model that is capable of performing well across all clients. In the ESL technique, each client trains its model separately by making use of data that is local to the client. An epoch on the local level is considered to be finished when all of the clients have completed one pass over their respective data samples. Following that, the central layers at the offloading server are averaged, and the client back layers are averaged at the client back model averaging server, which is responsible for averaging the client back model. This averaged client-back model is utilized to bring all of the client's local backside models up to date. Repeating these three steps—local epoch training, client back model averaging, and updating—continuously until the global/averaged model reaches a stable state known as the convergence state.

The personalization phase starts as soon as the global model converges. In this stage, personalized models are generated to accommodate the unique data distribution of each client. Only the client-side back layers are updated during this training phase, while the client front and all the server-side central layers remain frozen. Consequently, n distinct client back parts of the models are generated, corresponding to n distinct clients. Unlike the generalization phase, the personalization phase does not necessitate an aggregate step. To mitigate overfitting, we employ early stopping in this phase as well

$$key(x){:}Value(x) \quad (2a)$$

$$key(x) = generate\_unique\_id(client\_id, x\_id). \quad (2b)$$

B. Key-Value Store

ESL utilizes pretrained models and transfer learning to extract features and accelerate convergence. Additionally, it can help us reduce computation and communication overhead by allowing us to use a key-value store. To support this, we keep f_{CF} and f_{SF} constant in (1), the result will be the same each time the same input is passed. Consequently, we can compute it only once for each input and store it on the server-side key-value store as shown in (2a), (2b), and (3a), thereby reducing communication overhead.

Subsequently, we will proceed to train the server backend (f_{SB}) and client backend (f_{CB}) on our data set as shown in

$$\textit{Server-Side:} \quad value(key(x)) = f_{SF}(f_{CF}(x)) \tag{3a}$$

$$f(x) = \mathbf{f_{CB}}(\mathbf{f_{SB}}(value(key(x)))). \tag{3b}$$

During the generalization phase, the server-side key-value store caches a unique *key* per client per input as shown in (2b) and corresponding activation in *value* as shown in (3a). In batch training, samples are dynamically grouped into batches. In the initial epoch, each sample within a batch passes through the client front layers, generating activations for each sample. These activations, along with their unique keys, are transmitted to the server. The server-side center-front model processes these activations to generate further activations. Subsequently, the keys and their associated activations are stored in the server-side key-value store as per (4a). In subsequent epochs, there is no need to resend activations from the client front to the server, leading to enhanced communication efficiency by transmitting only the unique keys of samples to the server. This approach significantly reduces redundant computations and communication overhead during the generalization training phase

$$\textit{Client Side:} \ value(key(x)) = f_{SB}(f_{SF}(f_{CF}(x))) \tag{4a}$$

$$f(x) = \mathbf{f_{CB}}(value(key(x))). \tag{4b}$$

During the personalization phase, fine-tuning occurs solely in the client's back part of the model. Here, caching activations of each sample from the server-side center model to the client's local key-value store becomes possible, as illustrated in (4a) and (4b). This eliminates the need for communication between server and client, consequently reducing computational load and information exchange, ultimately enhancing performance in the overall training phase.

During the preprocessing step when the data set is limited, a common approach is to apply *random augmentations* to diversify the samples to create a more generalized model [44]. In Centralized Training, these random augmentations generate a different variety of samples every epoch. When we use the key-value store-based caching approach, we essentially remove the need to preprocess the original data set again. To achieve a better performing generalized model, we need to augment the data set again to generate more randomized samples useful for training. To solve this, we added a key-value store refresh. At certain intervals throughout the training process, the client regenerates the key-value stores to produce fresh samples for the model training. This allows for better generalization of the models across clients.

C. Client Back Side Model Customization

The objective is to optimize the function f_{CB} to reduce the computational load on the client side while ensuring high levels of model performance. To accomplish this, we have integrated customized functionalities for f_{CB} rather than exclusively depending on pretrained model components. Our study found that increasing client backside layers with heavy and complex f_{CB} can improve accuracy and personalize learning for non-IID data distribution. However, this method takes more computing and communication power. To solve this problem, we propose the idea of customized blocks on the client back side within SL to achieve comparable performance while offloading more of the model layers to the server. We explored different layer configurations on the client's backend and added those layers to improve the model capacity to capture complex patterns in non-IID data distributions.

IV. Experimental Setup

A. Experiment Goal

We have designed the experiments to answer the following questions:

- Q1: How do we select an appropriate model for SL, considering factors, such as model architecture, task requirements, and computational resources?
- Q2: How does varying the number of layers in the client's backside model affect SL performance and the computational workload for resource-constrained clients.
- Q3: How does customizing the model at the client back side affect SL performance and workload distribution between clients and servers?
- Q4: What is the overall performance of our proposed methods compared to other existing algorithms for image classification and 3-D image segmentation?
- Q5: Is the key-value store approach effective in ESL, and how much does it mitigate overhead in terms of communication and computation compared to other methods?

B. Experiment Setup

We have created our framework using PyTorch, ensuring a modular structure that permits flexible adjustments, including client count, DNN model, front/central/back layer quantities, and data points per client. To carry out our research, we operated on a workstation equipped with an Intel Xeon Gold 5218 CPU operating at 2.30 GHz, coupled with the NVIDIA RTX A6000 GPU. All experiments are repeated five times, and the average values are provided.

C. Data Set and Settings

Our research incorporates six distinct data sets, each presenting its own set of characteristics and difficulties. The information regarding the data sets can be found in Table I. To begin, we have the CIFAR-10 and FMNIST data set,

TABLE I
SUMMARY OF DATA SETS, NUMBER OF CLASSES, METRIC USED, TASK, BASE MODEL USED, AND PRETRAINED ON DATA SET INFORMATION

Dataset	Number of Classes	Metric Used	Task	Base Model	Pretrained On Dataset
CIFAR-10 [45] FMNIST [48]	10 10	Accuracy	Image Classification	MobileNetV3 [46] ResNet-18 [49]	ImageNet [47]
DR [50], [51] ISIC-2019 [31]	3 8	Balanced Accuracy [52]	Medical-Image Classification	ResNet-18 [49]	ImageNet [47]
KITS-19 [53], [54] IXI-Tiny [57], [58]	3 2	Dice-Score [55]	3D-Image Segmentation	nnUNet [30] 3D UNet [59]	MSD Pancreas [56] MSD Spleen [56]

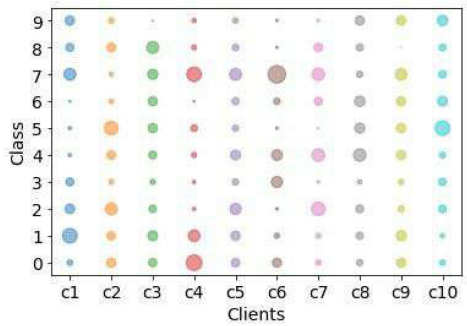

Fig. 2. CIFAR-10 data distribution among ten clients where larger circle indicates more number of training samples for that particular class and client.

which functions as the standard benchmark data set in the field of image classification. We established an experimental configuration comprising a total of ten clients. We used two settings in our study: 1) IID ensures fair evaluation and 2) scalability using equally distributed client data, while non-IID replicates real-world inequalities where we have used Dirichlet distribution to distribute the data among clients. The distribution of CIFAR-10 data among ten clients in a non-IID setting is illustrated in Fig. 2.

Following this, we investigate the Diabetic Retinopathy data set. Two data sets were utilized in this study: 1) EyePACS [51] and 2) APTOS [50]. The images contained in EyePACS were obtained from a variety of primary care sites in California and beyond, using a range of devices and operating under diverse conditions. In contrast, the APTOS data set was compiled from a variety of healthcare centers in India. To experiment, a configuration consisting of ten clients was established: the initial five clients (Client 1–Client 5) were supplied with the APTOS data set, while the subsequent five clients (Client 6–Client 10) were given the EyePACS data set. A distinct sample size of data was obtained for each client. To classify the images, the following labels were used: No DR (0), Moderate (1), and Severe (2). Every client executed the personalization phase to optimize the accuracy of their tests on their unique data sets. We have employed balanced accuracy as the metric in this instance.

Furthermore, the ISIC2019 [31] data set, consisting of dermoscopy images obtained from four hospitals, is utilized in our research. The data set under consideration comprises an image classification task with a specific emphasis on eight distinct melanoma classes. It is noteworthy that the label distribution exhibits a substantial class imbalance, thereby presenting an intriguing challenge. In addition, we modify the ISIC2019 data set to incorporate a six-client FL configuration, which replicates practical situations in which data is dispersed among numerous origins.

For our 3-D image segmentation experiments, we utilize the KITS-19 data set [53], [54], which was originally acquired from the Kidney Tumor Segmentation Challenge [53]. To create a federated version, we choose 6 clients from the data set provided by FLamby [31]. The segmentation task involves the identification of both kidneys and tumors, which are labeled as 1 and 2, respectively. Furthermore, for brain segmentation tasks, we employed the information extraction from image (IXI)-Tiny data set [57], [58] extracted from the IXIs database [60]. This data set provides T1-weighted magnetic resonance imaging (MRI) scans of the brain obtained from 3 hospitals. The federated version of this is provided by [31]. To address this task, we utilize pretrained models and refine them through fine-tuning to accomplish precise segmentation.

D. Metrics Used

The amount of data that must be transferred between a client and a server determines the cost of communication between the two parties. In FL, each client communicates only the trainable/unfrozen layer parameters to the server and receives the updated parameters in return. Conversely, in SL, the communication process involves multiple steps. Initially, activations are transmitted from the client front to the server. Subsequently, activations from the server to the client back side, along with gradients from the client back to the server, need to be sent. Moreover, our methodology involves the utilization of a distinct client model averaging servers. As a result, we are required to transmit the client back model to the server to obtain the updated averaged model. Additionally, as we utilize a key-value store, we must transport the keys from the client front to the server in a batch-processing manner. Furthermore, computational estimations on the client side are conducted by counting the floating point operations (FLOPs) for activation and gradient computation using the *THOP* [61] tool within PyTorch's built-in profiling function. Moreover, various performance metrics for a specific task and data set are detailed in Table I.

V. RESULTS AND DISCUSSION

In this section, we aim to address the questions outlined in the Experiment Goals section through the analysis of our experimental results.

TABLE II
PERFORMANCE COMPARISON BETWEEN RESNET-18 AND MOBILENETV3 (SMALL) ACROSS DIFFERENT DATA POINTS PER CLIENT IN IID SETTING ON CIFAR-10 WITH TEN CLIENTS

Model	Resnet-18	MobileNetV3(Small)
Number of Data points per client	Average Test Accuracy	Average Test Accuracy
50	76.10±0.19	74.35 ± 0.25
150	82.75±0.13	81.01 ± 0.25
250	83.79±0.26	82.79 ± 0.15
350	85.19±0.18	84.22 ± 0.16
500	88.17±0.13	85.46 ± 0.12

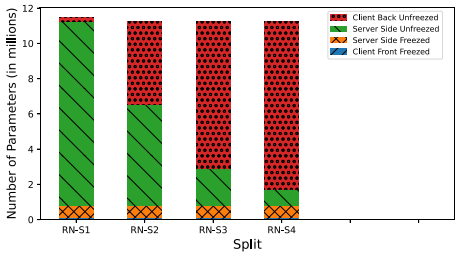

Fig. 3. Distribution of parameters across various split configurations of a ResNet-18 model. The client front freeze is at the bottom of each bar, with 5k parameters and therefore not visible.

A. Q1-Model Selection

For Q1, the evaluation involved two image classification models: 1) ResNet-18 and 2) MobileNetV3-Small, containing 11.7 million and 3 million parameters, respectively. Tests were conducted on the CIFAR-10 data set under an IID setup, ensuring uniform data distribution across clients. Both models were split such that only the last fully connected layer resided on the client's backside. The results presented in Table II illustrate that both models demonstrated improved accuracy with an increase in the number of data points per client. Notably, ResNet-18 consistently outperformed MobileNetV3-Small in terms of accuracy across varying data points per client. The variance in performance can be explained by the tradeoff between speed and accuracy inherent in MobileNetV3-Small. This model is specifically designed to prioritize smaller size and faster processing speed. In SL, the offloading capabilities to the server enable the utilization of complex models, in resource-constrained client-side devices. This allows for the selection of models based on task and performance requirements rather than computational limitations.

B. Q2—Effect of the Number of Layers in the Client-Back Side

In this section, we investigate the impact of varying the number of layers in the client-back side on both image classification and 3-D segmentation tasks. We recognize that the optimal number of layers may differ depending on the specific task and model architecture utilized. Therefore, we conduct separate analyses for image classification and 3-D segmentation to provide a comprehensive understanding.

1) Image Classification: We conducted an analysis using a ResNet-18 model and the CIFAR-10 data set to examine the effects of changing the number of layers on the client side and how they affect performance. For this, four different splits were used. The first convolutional layer of the ResNet-18 model was placed at the client's front in Split RN-S1, while the last linear layer was placed at the client's back. All remaining layers were offloaded on the server. The goal of this setup was to minimize the client's computing burden. Then, Splits RN-S2 through RN-S4 involved gradually moving more layers from the server to the client back while keeping the client front configuration unchanged. The number of layers on the client's back increased as a result of this change, increasing the client's effort for parameter updates.

TABLE III
PERFORMANCE METRICS (AVERAGE TEST ACCURACY AND EPOCH) FOR DIFFERENT SPLITS IN AN IID SETTING: E-1 (500 DATA POINTS) AND E-2 (50 DATA POINTS) ON CIFAR-10 DATA SET WITH TEN CLIENTS

Model Split	E-1 Test Accuracy	E-1 Epoch	E-2 Test Accuracy	E-2 Epoch
RN-S1	88.19	33	74.92	16
RN-S2	88.31	34	74.94	17
RN-S3	88.28	36	75.15	14
RN-S4	88.23	33	75.35	14

Notably, the total number of frozen and unfrozen layers remained unchanged in every split.

The statistical data summary of the number of parameters can be found in Fig. 3. Split RN-S2 shows an *8 times* increase in parameters on the client side as compared to Split RN-S1. This increase causes a significant spike in the overall number of operations needed, which increased by a factor of 274. In a similar vein, we have a *16 times* increase in parameters at the client side for Split RN-S3, which corresponds to a *476 times* increase in total operations. For Split RN-S4, the number of parameters at the client side has increased *18 times*, resulting in a *712 times* increase in the overall number of operations needed during training.

In experiments conducted under the IID setup, we observed no significant impact on the overall system performance, even with variations in model splits, as detailed in Table III. From this, we deduce that in scenarios where data is IID across clients, it may be feasible to offload a substantial portion of the computation to the server without compromising performance. To validate this assertion, we conducted a t-test to assess the statistical significance of RN-S3 compared to RN-S2. The obtained p-value was 0.162319, which exceeds the 0.05 significance level. Consequently, we infer that there is no statistically significant difference in performance between RN-S3 and RN-S2.

In the non-IID setting, the models split between the client and server had a significant impact on how well the entire model operated. As we transitioned from Split RN-S1 to RN-S4, a consistent pattern emerged in Table IV, showing that moving more layers to the client's backend notably enhanced the test accuracy of the Personalized Model. Moreover, it led to a reduction in the standard deviation across the accuracy of all clients. Furthermore, The average test accuracy improved

TABLE IV
PERFORMANCE METRICS (AVERAGE TEST ACCURACY, STANDARD DEVIATION, AND EPOCH) FOR DIFFERENT SPLITS IN A NON-IID SETTING FOR 500 DATA POINTS ON THE CIFAR-10 DATA SET WITH TEN CLIENTS

Model Split	Generalised Model			Personalised Model		
	Test Acc.	Std	Epoch	Test Acc.	Std	Epoch
RN-S1	82.3	4.21	29	86.65	2.43	25
RN-S2	82.2	3.15	25	87.51	2.48	23
RN-S3	82.1	3.12	26	88.55	2.37	21
RN-S4	82.4	3.11	27	**89.33**	**1.52**	18

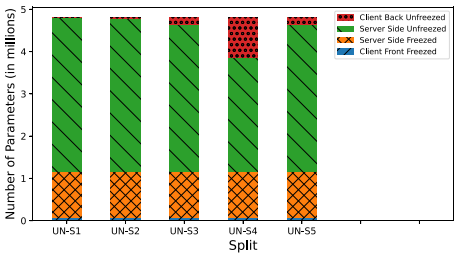

Fig. 4. Distribution of parameters across various segments of a MONAI 3-D UNet model in different splits for IXI-Tiny data set. The part that is not visible is having very less parameters.

significantly from 86.65% in Split RN-S1 to 89.33% in RN-S4, demonstrating the effectiveness of personalizing the model to client-specific data. To validate these findings, a t-test was conducted to assess significance by comparing Split RN-S2 to RN-S1. The resulting p-value, which is less than 0.00001, indicates statistical significance at the 0.05 significance level. Consequently, there exists a notable difference in performance between RN-S2 and RN-S1 based on the provided data. Furthermore, performance is fair across clients because it also reduces the standard deviation of test accuracy across all clients.

2) 3-D Image Segmentation: In a UNet architecture, the activation sizes decrease in the downsampling layers and increase back in the upsampling layers. Hence, it becomes crucial to split the model layers in such a way that the communication overhead is minimal, especially between the server-side back and client-side back models. With our key-value store approach, we now also need to account for the intermediate skip connection activations that are present in UNet models. The layers at the client-side back model are also compute-heavy as they process large activations. Hence, finding the optimal split is necessary such that both the communication and computation costs are minimized.

For IXI-Tiny, each of the 3 clients has about 100–200 samples. This in turn results in a fairly large key-value store dominating the 3-D UNet model size which only has around 4.8M parameters. We tried out five different splits as shown in Fig. 4 to observe the communication and computation overhead. We ensured a fair comparison by consistently applying identical hyperparameters across all the splits. The hyperparameters utilized for training include a training batch size of 16, a testing batch size of 10, the Adam optimizer, and a learning rate set to 0.001 for both client and server models across 10 epochs.

TABLE V
PERFORMANCE METRICS FOR DIFFERENT SPLITS ON IXI-TINY DATA SET FOR 3-D UNET MODEL. ONLY GLOBAL GENERALIZED MODEL RESULTS EXCEPT FOR UN-S5*, WHERE PERSONALIZATION AFTER GENERALIZATION HAS BEEN PERFORMED

Model Split	Total data transfer till convergence	Total GFLOPs till convergence	Test Dice Dice score ± std dev	Epochs
UN-S1	7,941,624,851	4.60	0.782±0.020	10
UN-S2	7,943,055,731	16.24	0.792±0.001	10
UN-S3	2,547,619,481	22.02	0.877±0.010	10
UN-S4	2,578,357,761	25.34	0.916±0.001	10
UN-S5	926,034,663	22.02	0.872±0.001	10
UN-S5*	926,034,663	25.02	0.876±0.001	15

Table V presents a comparison of performance among the different splits for IXI-Tiny. We started with UN-S1 with only 1845 parameters on the client's backside. It was computationally lightweight on the client side but incurred high-communication costs due to large activation sizes. In UN-S2, we increased the size of the client-side back model, leading to higher computation with a slight improvement in Dice score. In UN-S3 we further shifted more layers to the client back model and saw a 12% increase in Dice score but 4.7 times higher computation compared to UN-S1. However, UN-S3 significantly reduced communication costs (3.2 times) due to the transfer of smaller sized activations from the server to the client back. Moving to UN-S4, we enlarged the client-back model by incorporating all decoder part of the model, resulting in a 3.19 times reduction in communication and a 5.4 times increase in computation from UN-S1, with a 17% Dice Score improvement.

Furthermore, UN-S5 is derived from UN-S3 but with only the client-side back model trainable while central layers were frozen. This allowed us to cache activations on the client side, requiring communication only for model averaging. Although there was a slight reduction in the Dice Score due to fewer trainable parameters, it remained close to UN-S3. After acquiring the generalized model in UN-S5, we proceeded with personalized training but found no improvement in performance, as reported by UN-S5*. In UN-S5*, the Dice score stayed constant, and the process demanded more computation due to the inclusion of extra epochs in the personalization phase.

In summary, assigning more layers to the client's backside improved the Dice Score, but it is computationally demanding for the client. The ideal split depends on the available resources; for instance, UN-S4 performs best when clients have ample resources, whereas UN-S5 is preferable for balancing computation and communication efficiency. Additionally, our findings indicate that personalization does not lead to improved performance in 3-D segmentation tasks.

C. Q3: Effect of Customization of Model

The results from the previous section for image classification indicate that increasing the number of layers on the client's backend (transitioning from RN-S1 to RN-S3)

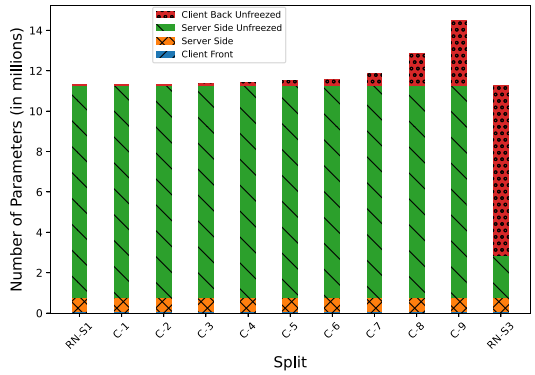

Fig. 5. Distribution of parameters across various segments of a ResNet-18 model in different customized splits. The client front freeze is at the bottom of each bar, with 5k parameters and therefore not visible.

TABLE VI
COMPARISON OF PERFORMANCE METRIC FOR DIFFERENT CONFIGURATIONS OF CLIENT BACK ACROSS VARIOUS SPLITS ON CIFAR-10 DATA SET WITH TEN CLIENTS. THE BEST RESULT IS UNDERLINED. LL: LINEAR LAYER, CL: CONVOLUTION LAYER, DS: DEPTHWISE SEPARABLE LAYER, AND CB: CONVOLUTION BLOCK

Split	Client Back Side	Number of parameters	CIFAR-10 ESL-G	CIFAR-10 ESL-P
RN-S1	**LL(512x10)**	**5,130**	**82.39**	**86.65**
C-1	2LL(512x64X10)	33,482	81.65	87.75
C-2	DS(64x5x5)+LL(64X10)	38,730	82.9	86.4
C-3	DS(128x5x5)+2LL(128x64X10)	79,946	83.1	86
C-4	DS(256x5x5)+2LL(256x64X10)	154,058	83.1	86.9
C-5	**DS(512x5x5)+LL(512x10)**	273,930	84.02	88.36
C-6	CL(64x5x5)+LL(64x10)	295,690	82.6	86.8
C-7	CL(128x5x5)+2LL(128x64X10)	598,986	81.8	87.3
C-8	CL(64x3x3)+LL(64x10)	1,606,410	84.00	86.3
C-9	CL(128x3x3)+2LL(128x64X10)	3,220,426	83.62	86.30
RN-S3	**Last CB+LL(512x10)**	**8,398,858**	**82.34**	**88.52**

improves the personalized model for each client and subsequently increases test accuracy. However, this can significantly increase the computational burden on the client's backend, potentially by up to 476 times. To address this, it becomes essential to explore custom models on the client's backend that strike a balance between parameter count and accuracy. To achieve this, we experimented with various customized blocks from Configurations C-1 to C-9.

Fig. 5 shows the distribution of the parameters between client and server in various splits and configurations. From Table VI, it is evident that certain customizations led to improvements in performance compared to RN-S3 while keeping the parameter count in check. For instance, configuration C-5, involving depthwise-separable layers followed by a Linear Layer, achieved competitive test accuracies for CIFAR-10 compared to RN-S3 while maintaining a lower parameter count, approximately 300 times less than RN-S3. Furthermore, in Table VII we are showing the performance of customization on all data sets. These results indicate that by carefully selecting and designing layers for the client's backside, it is possible to achieve comparable or even improved performance compared to a heavy, noncustomized model on the client's backside. In conclusion, customization enables a greater offload of a model to the server side while preserving accuracy and performance.

D. Q4: Overall Performance

In this section, we aim to compare the overall performance of our proposed methods with other existing algorithms for image classification and 3-D image segmentation.

1) Image Classification: To address Q4, we have used Fedavg [32], SL [6], SLV2 [35] as baseline algorithms. We have implemented these with the ResNet-18 model architecture with random initialization of weights. Also we have used FedAvg_TL, FedProx_TL [33], Scaffold_TL [34] These algorithms we have implemented with the ResNet-18 model with pretrained weights with the same number of frozen and unfrozen layers that we have used in ESL. Apart from these algorithms, for performance comparison, we also include a centralized model assuming that it has access to the labeled data from all the clients. It is expected to have the highest performance but can compromise data privacy. Here, for the centralized model (Pooled_TL), again for fair comparison, we have used the ResNet-18 model with pretrained weights with the same number of frozen and unfrozen layers that we have used in ESL. In our approach we have evaluated the performance after generalization with a generalized model (ESL-G) and after personalization with the personalized model (ESL-G).

Furthermore, we have also used UN-G and UN-P, where we have used ResNet-18 pretrained weights, but we have unfrozen all the layers for generalization and personalization in ESL training. For the extreme case, we also included NF-G and NF-P, wherein we utilized the baseline ResNet-18 pretrained weights with adjustments made to the last layer based on the data set requirements.

Table VIII highlights the consistent superiority of our ESL-G and ESL-P models across multiple data sets compared to existing algorithms. NF-G and NF-P exhibit lower accuracy compared to ESL-G and ESL-P, indicating the limitations of solely relying on pretraining without fine-tuning. This approach faces challenges in capturing specific data set details, leading to poorer performance. Additionally, unfreezing all layers (UN-G and UN-P) may increase slightly accuracy for data sets within the same domain type, such as CIFAR-10, which shares similarities with the ImageNet-trained model. However, this approach tends to lead to overfitting when applied to data sets from different domains, as seen with ISIC-2019.

A convergence plot of validation accuracy and training loss for different training methods are shown in Fig. 6. ESL has better validation accuracy convergence than other training methods on CIFAR-10 and ISIC-2019 data sets. We have faster and smoother convergence than competing techniques. All loss curve slopes increase significantly when personalization begins, indicating accelerated convergence. Personalization adapts to local data distribution, proving its efficiency in model training. In summary, our ESL-G and ESL-P models showcase superior performance compared to existing FL and SL algorithms across various data sets.

TABLE VII
COMPARISON OF PERFORMANCE METRIC FOR DIFFERENT CONFIGURATIONS OF CLIENT BACK ACROSS VARIOUS SPLITS ON VARIOUS DATA SETS (EACH WITH TEN CLIENTS). THE BEST RESULT IS UNDERLINED. LL: LINEAR LAYER, CL: CONVOLUTION LAYER, AND DS: DEPTHWISE SEPARABLE LAYER

Split	Client Back Side	Number of parameters	CIFAR-10		FMNIST		ISIC		DR	
			ESL-G	ESL-P	ESL-G	ESL-P	ESL-G	ESL-P	ESL-G	ESL-P
RN-S1	**LL(512x10)**	**5,130**	**82.39**	**86.65**	**82.46**	**91.48**	**61.90**	**68.97**	**50.87**	**56.50**
C-1	2LL(512x64X10)	33,482	81.65	87.75	82.02	89.72	61.67	69.0	52.9	56.98
C-5	**DS(512x5x5)+LL(512x10)**	273,930	84.02	88.36	84.47	90.53	61.13	71.40	53.46	61.30
RN-S3	**Last CB+LL(512x10)**	**8,398,858**	**82.34**	**88.52**	**82.55**	**88.97**	**66.16**	**74.43**	**57.6**	**63.97**

TABLE VIII
PERFORMANCE COMPARISON ACROSS DIFFERENT METHODS ON VARIOUS DATA SETS WITH NON-IID SETTING AND TEN CLIENTS. WITHOUT CLIENT BACK MODEL CUSTOMIZATION VERSION OF ESL IS SHOWN AS ESL°

Method	CIFAR-10	FMNIST	ISIC	DR
ESL-G	84.02	84.47	61.13	53.46
ESL-P	88.36	91.53	71.40	61.30
Pooled_TL	**89.12**	**91.88**	**75.50**	**65.4**
ESL-G°	**82.39**	**82.46**	**61.90**	**50.87**
ESL-P°	**86.65**	**91.48**	**68.97**	**56.50**
NF-G	**70.25**	**68.57**	**49.21**	**41.74**
NF-P	**78.97**	**83.65**	**59.82**	**46.51**
UN-G	**85.15**	**85.57**	**55.30**	**46.56**
UN-P	**87.11**	**91.95**	**60.11**	**49.32**
FedAvg	61.01	72.42	48.95	40.21
SL	65.42	76.22	51.95	41.45
SLV2	65.23	76.80	49.78	41.25
FedAvg_TL	67.60	78.71	54.11	42.05
FedProx_TL	67.06	72.51	55.22	42.06
Scaffold_TL	72.01	77.33	54.82	46.42

2) 3-D Segmentation: For Q4, we utilized baseline results from FLamby's [31] experiments involving different training methods, such as FedAvg, FedProx, and Scaffold. For the KITS-19 data set, where there were limited samples per client, we employed a strategy to enhance the training data by doubling the training samples with strong random augmentations on each sample. This augmentation technique, combined with refreshing the key-value store periodically (after every 5 epochs) to update the augmented samples, contributed to a more robust ESL setup with improved generalization. The other hyperparameters include training batch size 4, testing batch size 2, Adam optimizer, and learning rate equals 0.0006 for all client and server models. Consequently, leveraging strong augmentations and extended training duration, alongside our pretrained model within the same domain, resulted in a remarkable 45% enhancement over the results obtained using FedAvg in FLamby [31]. Moreover, our experiments on the IXI-Tiny data set demonstrated significantly better performance compared to various algorithms, yielding an average improvement of 17% over FLamby's results, as shown in Table IX.

In summary, our methodology, characterized by the strategic application of augmentations and the utilization of pretrained models, yielded significant performance enhancements in 3-D Segmentation tasks.

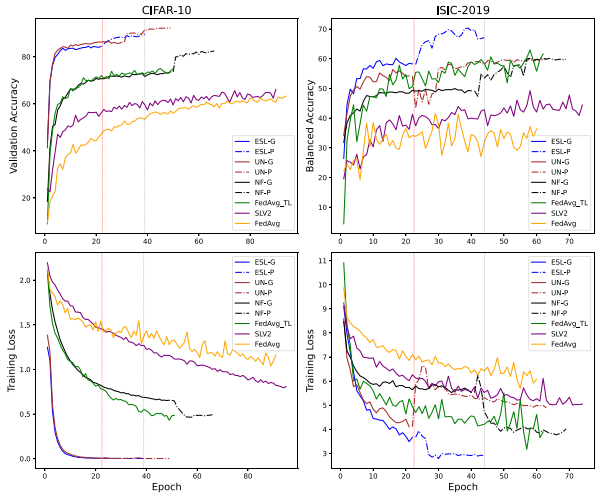

Fig. 6. Convergence curves for training loss and validation performance for CIFAR-10 and ISIC-2019 using various methodologies. The red and gray vertical lines indicate the start of personalization and the end of ESL training, respectively.

TABLE IX
PERFORMANCE COMPARISON ACROSS DIFFERENT METHODS [31] ON KITS-19 AND IXI-TINY DATA SETS

Dataset	ESL	Pooled	FedAvg	FedProx	Scaffold
KITS-19	**0.746**	0.626	0.514	0.561	0.553
IXI-Tiny(UN-S5)	**0.872**	0.954	0.769	0.768	0.769

E. Q5: Evaluation of Key-Value Store

In this section, we evaluate the effectiveness of the key-value store-based caching approach in reducing communication and computation overhead compared to traditional methods.

1) Image Classification: To analyze the impact on the image classification task, the evaluation is broken down into specific aspects as follows.

1) *Communication Reduction:* When compared to other conventional FL/SL approaches, Table X shows the communication reduction achieved with our methodology. Importantly, the amount of data that needs to be transferred is proportional to the sample size of the data points linked to each client and the size of the model. The use of ESL leads to a huge decrease in

TABLE X
COMMUNICATION OVERHEAD (IN BYTES) OF VARIOUS METHODS ON CIFAR-10 DATA SET WITH NON-IID SETTING PER CLIENT. ESL$^-$ IS WITHOUT KEY-VALUE STORE VERSION OF ESL

Method	Epochs till convergence	Client Front to Server (Activation)	Key in KV Store	Server to/from Client Back (Activation+gradient)	Model Weights to Server	Total data-transfer till convergence	Reduction compared to other method
ESL-G	22	401, 408, 000	22, 000	45, 056, 000	902, 880	447, 388, 880	–
ESL-P	20	0	0	0	0	0	–
Total ESL	42	401, 408, 000	22, 000	45, 056, 000	902, 880	447, 388, 880	–
ESL-G$^-$	22	8, 830, 976, 000	0	45, 056, 000	902, 880	8, 876, 934, 880	–
ESL-P$^-$	20	8, 028, 160, 000	0	20, 480, 000	0	8, 048, 640, 000	–
Total ESL$^-$	42	16, 859, 136, 000	0	65, 536, 000	902, 880	16, 925, 574, 880	**37.8X**
UN-G	30	24084480000	0	61440000	2288640	24148208640	–
UN-P	18	14450688000	0	36864000	0	14487552000	–
Total UN	48	38535168000	0	98304000	2288640	38635760640	**86.35X**
FL	94	0	0	0	8, 408, 594, 784	8, 408, 594, 784	**18.8X**
FL_TL	50	0	0	0	3, 359, 543, 200	3, 359, 543, 200	**7.5X**
SL	92	72, 253, 440, 000	0	188, 416, 000	3, 775, 680	72, 437, 760, 000	**161.9X**

communication due to the elimination of client front-side model updates and key-value stores. Also since only the client back side model is updated during the personalization phase of training, the equivalent communication reduction for CIFAR10 is by 37.8 times. An additional noteworthy finding is that the reduction attained for ESL surpasses that of conventional SL by 161.9 times. As illustrated in Fig. 6, this is because transfer learning and personalization accelerate convergence during training. ESL reduces communication by 18.8 times in comparison to FL, in which the entire model must be transmitted to the server. In contrast, ESL reduces communication by 7.5 times in comparison to FL with transfer learning. Furthermore, ESL reduces communication by 86.35 times in comparison to ESL with all layers unfreezed because it will not help us to leverage caching, and also convergence time will be high. Furthermore, Table XI provides a comparative analysis of communication overhead reduction across all data sets, demonstrating the effectiveness of our approach compared to various methods.

2) *Computation Reduction:* The computational statistics are examined in Table XII, which draws attention to the floating-point operations performed at the client end for different approaches on the CIFAR-10 data set using the ResNet-18 model. Since we do not have to recompute the same activation when using the pretrained model, ESL results in a 41.78 times reduction in calculation time when compared to ESL without the key-value store. FL requires 4219.5 times as many computations as ESL, FL with transfer learning requires 1423.9 times as much computational work, and SL approaches require around 274.4 times as much computational work as ESL, indicating that both FL and SL methods demand much more computations at the client. Additionally, unfreezing all layers in ESL leads to a computation increase of 143.24 times compared to the original ESL version. Furthermore, Table XIII provides a comparative analysis of computation overhead reduction across all data sets, demonstrating the effectiveness of our approach compared to various methods.

TABLE XI
REDUCTION IN COMMUNICATION OVERHEAD OF ESL WITH RESPECT TO DIFFERENT TECHNIQUES ACROSS ALL DATA SETS. WITHOUT KEY-VALUE STORE VERSION OF ESL IS SHOWN AS ESL$^-$

Method	CIFAR-10	FMNIST	ISIC	DR
FL	18.8	8.21	1.94	2.94
FL_TL	7.51	5.21	1.16	1.80
SL	161.9	93.7	109.3	109.7
ESL$^-$	37.8	40.1	40.4	42.5

2) 3-D Image Segmentation: In 3-D image segmentation tasks, we achieved a remarkable 45× reduction in computation for the KITS-19 data set, which is particularly significant given the computational demands inherent in segmentation tasks. Furthermore, the KITS-19 data set only had an average of 20 samples per client, and the size of the key-value store was quite small. The communication of ESL was mainly dominated by the size of the client-back side model shared for model averaging. Hence, communication costs remained high. Based on this observation, a tradeoff between computation and communication was discerned. When clients have sufficient computational resources, it may be more advantageous to utilize a split configuration for 3-D segmentation tasks to reduce communication. On the other hand, clients utilizing devices with limited resources may give precedence to split configurations that reduce computation demands, although this may result in higher data transfers.

To address the issue of high communication and computation overhead in 3-D segmentation tasks, we proposed combining our key-value store technique with FL and transfer learning. It is important to acknowledge that integrating the key-value store technique into any distributed training method requires the utilization of a pretrained model containing specific frozen layers. Furthermore, Table XIV shows that we significantly reduced communication and computation costs by

TABLE XII
FLOATING-POINT OPERATIONS AT THE CLIENT IN VARIOUS METHODS ON CIFAR-10 DATA SET WITH NON-IID SETTING PER CLIENT. WITHOUT KEY-VALUE STORE VERSION OF ESL IS SHOWN AS ESL$^-$

Method	Total Epoch till convergence	Client Side Front Layer	Server Side Central Layer	Client Side Back Layer	Total GLOPS till convergence	Reduction compared to other method
ESL-G	22	0.242	0	0.000676	0.243126	–
ESL-P	20	0	0	0.000614	0.000614	–
Total ESL	42	0.242	0	0.001290	0.243740	–
ESL-G$^-$	22	5.333	0	0.000676	5.334585	–
ESL-P$^-$	20	4.849	0	0.000614	4.849623	–
Total ESL$^-$	42	10.18	0	0.001290	10.18421	**41.78X**
Total UN	48	21.82	0	0.001474	34.91433	**143.24X**
FL	94	68.37	960.1	0.002887	1028.469	**4219.5X**
FL_TL	50	12.12	334.9	0.001536	347.0689	**1423.9X**
SL	92	66.92	0	0.002826	66.91914	**274.4X**

TABLE XIII
REDUCTION IN GFLOPS OF ESL WITH RESPECT TO DIFFERENT TECHNIQUES ACROSS ALL DATA SETS. WITHOUT KEY-VALUE STORE VERSION OF ESL IS SHOWN AS ESL$^-$

Method	CIFAR-10	FMNIST	ISIC	DR
FL	4219.5	2915.9	3499.6	3364.1
FL_TL	1423.9	1565.3	1765.0	1735.8
SL	274.55	164.03	181.97	184.88
ESL$^-$	41.783	46.727	44.751	47.715

TABLE XIV
REDUCTION IN COMMUNICATION AND COMPUTATION OVERHEAD OF ESL WITH RESPECT TO FEDAVG [31]

Dataset	Technique	Communication Reduction	Computation Reduction
KITS-19	ESL	2.38	45.43
IXI-Tiny	ESL(UN-S5)	0.20	2.34
IXI Tiny	FL_TL + KV	26.04	2.21

TABLE XV
COMPARISON OF ACCURACY AND COMPRESSED SIZE OF ESL WITH [13]. WE ASSUME THE ORIGINAL COMMUNICATION SIZE (NONCOMPRESSION CASE) TO BE 100

Method	Accuracy (%)	Compressed Size
TOP-K Sparsification [13]	66.01	12.5
ESL-P	**75.1**	**1.21**

TABLE XVI
COMPARISON OF BALANCED ACCURACY AND STANDARD DEVIATION ESL WITH FLAMBY [31]

Method	Balanced Accuracy(%)	Standard Deviation
FedAvg [31]	58.06	13.19
FedProx [31]	55.02	13.21
Scaffold [31]	61.9	13.90
ESL-G	61.1	13.18
ESL-P	**71.4**	**11.62**

caching activations from frozen layers and broadcasting only trainable parameters for averaging. This approach provides a promising option for mitigating the constraints faced by communication-intensive tasks, such as 3-D segmentation.

In summary, ESL achieves substantial reductions in communication and computational overhead at the client end, by efficiently utilizing a key-value cache. These computational savings contribute to enhanced efficiency and scalability of distributed learning systems, making our approach highly promising for practical deployment in real-world scenarios.

F. Comparison With Some Recent Approaches

The objective of this experiment is to assess the efficacy of our suggested technique compared to existing methods in minimizing communication overhead while enhancing accuracy. We conducted experiments using the CIFAR-100 data set and compared our results with a recent approach [13]. From the analysis presented in Table XV, it is evident that our method outperforms the existing approach, achieving a 13% better accuracy with reduced communication overhead during the training phase.

Furthermore, we expanded our comparative analysis to include the ISIC data set, where we compared our method against FLamby [31]. We assessed both the mean balanced accuracy and the standard deviation of balanced accuracy across all clients. The presence of model bias among clients implies a high heterogeneity, with some clients outperforming others, leading to a high-standard deviation in their balanced accuracy. Table XVI shows that our method yields lower standard deviation with significant performance enhancements.

The results emphasize the effectiveness of our suggested approach in minimizing communication overhead and improving overall performance on various data sets, hence showcasing its potential for practical use in real-world situations.

VI. CONCLUSION

This article presents ESL, a novel approach designed to address the challenges of distributed model training on IoT devices. ESL achieves notable enhancements over conventional FL and SL approaches by employing innovative methods like key-value store-based caching and customized model strategies. Our experimental results demonstrate remarkable

gains in various metrics. First, ESL achieves a substantial reduction in communication traffic during training, with improvements ranging from 3.92× to 100× compared to baseline FL and SL methods, respectively. Second, ESL significantly reduces computations during training, achieving a remarkable 1623× reduction compared to baseline FL methods on resource-constrained devices. Moreover, ESL maintains high-accuracy levels while reducing communication and computation overhead, surpassing baseline FL with transfer learning by an average of 35% and baseline SL by an average of 34.8%. Overall, ESL offers a practical and scalable solution for distributed learning on IoT devices, addressing the unique challenges posed by non-IID and medical data. As the demand for distributed learning on IoT devices continues to grow, ESL provides a robust framework for addressing the evolving needs of real-world applications.

REFERENCES

[1] N. C. Thompson, K. Greenewald, K. Lee, and G. F. Manso, "The computational limits of deep learning," 2020, *arXiv:2007.05558*

[2] B. Norgeot, B. S. Glicksberg, and A. J. Butte, "A call for deep-learning healthcare," *Nat. Med.*, vol. 25, pp. 14–15, Jan. 2019, doi: 10.1038/s41591-018-0320-3.

[3] G. Sreenivasulu, N. Thulasi Chitra, S. Viswanadha Raju, and V. M. Kuthadi, "Deep learning-based smart surveillance system," in *Smart Trends in Computing and Communications*, Y.-D. Zhang, T. Senjyu, C. So-In, and A. Joshi, Eds., Singapore: Springer Nat., 2023, pp. 111–123. [Online]. Available: https://dx.doi.org/10.1007/978-981-16-9967-2_12

[4] Y. Shi, X. Xue, Y. Qu, J. Xue, and L. Zhang, "Machine learning and deep learning methods used in safety management of nuclear power plants: A survey," in *Proc. Int. Conf. Data Min. Workshops (ICDMW)*, 2021, pp. 917–924.

[5] B. McMahan, E. Moore, D. Ramage, S. Hampson, B. A. Y. Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. 20th Int. Conf. Artif. Intell. Stat.*, 2023, pp. 1273–1282.

[6] P. Vepakomma, O. Gupta, T. Swedish, and R. Raskar, "Split learning for health: Distributed deep learning without sharing raw patient data," 2018, *arXiv:1812.00564*.

[7] H. Sun et al., "FedLALR: Client-specific adaptive learning rates achieve linear speedup for non-IID data," 2023, *arXiv:2309.09719*.

[8] A. F. Aji and K. Heafield, "Sparse communication for distributed gradient descent," 2017, *arXiv:1704.05021*.

[9] S. M. Shah and V. K. N. Lau, "Model compression for communication efficient federated learning," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 9, pp. 5937–5951, Sep. 2023.

[10] S. Han, J. Pool, J. Tran, and W. Dally, "Learning both weights and connections for efficient neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2015, pp. 1–9.

[11] P. Han, S. Wang, and K. K. Leung, "Adaptive gradient sparsification for efficient federated learning: An online learning approach," 2020, *arXiv:2001.04756*.

[12] D. Alistarh, D. Grubic, J. Li, R. Tomioka, and M. Vojnovic, "QSGD: Randomized quantization for communication-optimal stochastic gradient descent," 2016, *arXiv:1610.02132*.

[13] F. Zheng, C. Chen, L. Lyu, and B. Yao, "Reducing communication for split learning by randomized top-k sparsification," 2023, *arXiv:2305.18469*.

[14] D. Chai et al., "Federated singular vector decomposition," 2021, *arXiv:2105.08925*.

[15] S. U. Stich, J.-B. Cordonnier, and M. Jaggi, "Sparsified SGD with memory," 2018, *arXiv:1809.07599*.

[16] W. Wen et al., "TernGrad: Ternary gradients to reduce communication in distributed deep learning," 2017, *arXiv:1705.07878*.

[17] F. Sattler, S. Wiedemann, K.-R. Müller, and W. Samek, "Sparse binary compression: Towards distributed deep learning with minimal communication," 2018, *arXiv:1805.08768*.

[18] F. Sattler, S. Wiedemann, K.-R. Müller, and W. Samek, "Robust and communication-efficient federated learning from non-IID data," 2019, *arXiv:1903.02891*.

[19] H. Wang, S. Sievert, S. Liu, Z. Charles, D. Papailiopoulos, and S. Wright "ATOMO: Communication-efficient learning via atomic sparsification," 2018, *arXiv:1806.04090*

[20] T. Vogels, S. P. Karimireddy, and M. Jaggi, "PowerSGD: Practical low-rank gradient compression for distributed optimization," 2019, *arXiv:1905.13727*.

[21] X. Chen, J. Li, and C. Chakrabarti, "Communication and computation reduction for split learning using asynchronous training," in *Proc. IEEE Workshop Signal Process. Syst. (SiPS)*, 2021, pp. 76–81.

[22] A. Ayad, M. Renner, and A. Schmeink, "Improving the communication and computation efficiency of split learning for IoT applications," in *Proc. IEEE Global Commun. Conf. (GLOBECOM)*, 2021, pp. 1–6.

[23] Z. Zhou et al., "A novel optimized asynchronous federated learning framework," in *Proc. IEEE 23rd Int Conf. High Perform. Comput. Commun.*, 2021, pp. 2363–2370.

[24] Y. Kanamori, Y. Yamasaki, S. Hosoai, H. Nakamura, and H. Takase, "An asynchronous federated learning focusing on updated models for decentralized systems with a practical framework," in *Proc. IEEE 47th Annu. Comput., Softw., Appl. Conf. (COMPSAC)*, 2023, pp. 1147–1154.

[25] D. Novoa-Paradela, O. Fontenla-Romero, and B. Guijarro-Berdiñas "Fast deep autoencoder for federated learning," *Pattern Recognition*, vol. 143, Nov. 2023, Art. no. 109805.

[26] M. Beitollahi and N. Lu, "FLAC: Federated learning with autoencoder compression and convergence guarantee," in *Proc. IEEE Global Commun. Conf.*, 2022, pp. 4589–4594.

[27] S. Becker, K. Styp-Rekowski, O. V. L. Stoll, and O. Kao, "Federated learning for autoencoder-based condition monitoring in the Industrial Internet of Things," in *Proc. Int. Conf. Big Data*, 2022, pp. 5424–5433.

[28] B. Neyshabur, H. Sedghi, and C. Zhang, "What is being transferred in transfer learning?" 2020, *arXiv:2008.11687*.

[29] M. Wadhwa, G. R. Gupta, A. Sahu, R. Saini, and V. Mittal, "PFSL: Personalized & fair split learning with data & label privacy for thin clients," 2023, *arXiv:2303.10624*

[30] F. Isensee, P. F. Jaeger, S. A. A. Kohl, J. Petersen, and K. H. Maier-Hein, "nnU-Net: A self-configuring method for deep learning-based biomedical image segmentation," *Nat. Methods*, vol. 18, pp. 203–211, Feb. 2021.

[31] J. Ogier du Terrail et al., "Flamby: Datasets and benchmarks for cross-silo federated learning in realistic healthcare settings," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 35, 2022, pp. 5315–5334.

[32] H. B. McMahan, E. Moore, D. Ramage, and B. A. Y. Arcas, "Federated learning of deep networks using model averaging," 2016, *arXiv:1602.05629*.

[33] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," in *Proc. Mach. Learn. Syst.*, 2020, pp. 1–22.

[34] S. P. Karimireddy, S. Kale, M. Mohr, S. J. Reddi, S. U. Stich, and A. T. Suresh, "SCAFFOLD: Stochastic controlled averaging for federated learning," 2021, *arXiv:1910.06378*.

[35] C. Thapa, M. A. P. Chamikara, S. Camtepe, and L. Sun "SplitFed: When federated learning meets split learning," 2022, *arXiv:2004.12088*.

[36] Y. Oh, J. Lee, C. G. Brinton, and Y.-S. Jeon "Communication-efficient split learning via adaptive feature-wise compression," 2023, *arXiv:2307.10805*.

[37] S. Lyu, Z. Lin, G. Qu, X. Chen, X. Huang, and P. Li "Optimal resource allocation for U-shaped parallel split learning," in *Proc. IEEE Globecom Workshops (GC Wkshps)*, 2023, pp. 197–202.

[38] D. J. Bajpai, V. K. Trivedi, S. L. Yadav, and M. K. Hanawal, "SplitEE: Early exit in deep neural networks with split computing," 2023, *arXiv:2309.09195*.

[39] A. Bakhtiarnia, N. Miloševic, Q. Zhang, D. Bajovic, and A. Iosifidis, "Dynamic split computing for efficient deep edge intelligence," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, 2022, pp. 1–5.

[40] V. Smith, C. K. Chiang, M. Sanjabi, and A. S. Talwalkar, "Federated multi-task learning," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 1–11.

[41] Y. Zhao, M. Li, L. Lai, N. Suda, D. Civin, and V. Chandra, "Federated learning with non-IID data," 2018, *arXiv:1806.00582*.

[42] T. Yu, E. Bagdasaryan, and V. Shmatikov, "Salvaging federated learning by local adaptation," 2020, *arXiv:2002.04758*.

[43] B. Neyshabur, H. Sedghi, and C. Zhang, "What is being transferred in transfer learning?" in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 512–523.

[44] J. Xu, M. Li, and Z. Zhu, "Automatic data augmentation for 3d medical image segmentation," in *Proc. Int. Conf. Med. Image Comput. Comput.-Assist. Interv.*, 2020, pp. 378–387.

[45] A. Krizhevsky, "Learning multiple layers of features from tiny images," Dept. Comput. Sci., Univ. Toronto, Toronto, ON, Canada, Rep. TR-2009, 2009.

[46] A. Howard et al., "Searching for MobileNetV3," 2019, *arXiv:1905.02244*.

[47] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *Proc. IEEE Conf. Comput. Vis. Pattern Recogn.*, 2009, pp. 248–255.

[48] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-MNIST: A novel image dataset for benchmarking machine learning algorithms," 2017, *arXiv:1708.07747*.

[49] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Las Vegas, NV, USA, 2016, pp. 770–778, doi: 10.1109/CVPR.2016.90.
[50] 2019, "Aptos 2019 diabetic retinopathy dataset," Dataset, Kaggle. [Online]. Available: https://academictorrents.com/details/d8653db45e7f111dc2c1b595bdac7ccf695efcfd
[51] "Kaggle DR dataset (EyePACS)," Dataset. Kaggle. Accessed: Mar. 18, 2023. [Online]. Available: https://paperswithcode.com/dataset/kaggle-eyepacs
[52] M. Grandini, E. Bagli, and G. Visani, "Metrics for multi-class classification: an overview," 2020, *arXiv:2008.05756*.
[53] N. Heller et al., "The state of the art in kidney and kidney tumor segmentation in contrast-enhanced ct imaging: Results of the KiTS19 challenge," *Med. Image Anal.*, vol. 67, Jan. 2020, Art. no. 101821.
[54] N. Heller et al., "The KiTS19 challenge data: 300 kidney tumor cases with clinical context, CT semantic segmentations, and surgical outcomes," 2019, *arXiv:1904.00445*.
[55] L. R. Dice, "Some factors affecting the distribution of the prairie vole, forest deer mouse, and prairie deer mouse," *Ecology*, vol. 3, no. 1, pp. 29–47, 1992.
[56] M. Antonelli et al., "The medical segmentation decathlon," *Nat. Commun.*, vol. 13, p. 4128, Jul. 2022.
[57] F. Pérez-García, R. Sparks, and S. Ourselin, "TorchIO: A python library for efficient loading, preprocessing, augmentation, and patch-based sampling of medical images in deep learning," *Comput. Methods Programs Biomed.*, vol. 208, Sep. 2021, Art. no. 106236.
[58] "Ixitiny dataset," Dataset. Accessed: May 18, 2022. [Online]. Available: https://torchio.readthedocs.io/datasets.html#torchio.datasets.ixi.IXITiny.
[59] M. J. Cardoso. "MONAI model zoo." Accessed: Jun. 16, 2023. [Online]. Available: https://monai.io/model-zoo.html.
[60] "Brain development team," IXI dataset. Accessed: Feb. 2, 2022. [Online]. Available: https://brain-development.org/ixi-dataset/
[61] "THOP: Pytorch-opcounter." 2017. [Online]. Available: https://github.com/Lyken17/pytorch-OpCounter

Manisha Chawla received the Master of Technology degree in computer science and engineering from Indian Institute of Technology Bhilai, Bhilai, India, in May 2024.

She is currently a Lecturer with the Department of Computer Science and Engineering, Government Girls Polytechnic, Bilaspur, India. Her current research interests include distributed and edge machine learning, federated learning, split learning, medical imaging, and graph neural networks.

Gagan Raj Gupta received the Ph.D. degree in computer science and engineering from Purdue University, West Lafayette, IN, USA, in 2009.

He then spent ten years with AT&T Labs, Inc., San Ramon, GA, USA as the Lead Architect for new technology development in monitoring and probing AT&T's mobility (3G/4G/5G) and IMS physical and virtual network functions. He is currently an Associate Professor with the Department of Computer Science and Engineering, Indian Institute of Technology Bhilai, Bhilai, India. His research expertise includes large language models for networks, distributed GNNs, federated/split learning, multimodal learning and medical applications, automated root cause analysis using time series and/or video data, automated surveillance systems, algorithm design, and optimization.

Shreyas Gaddam received the B.Tech. degree in computer science from Shri Shankaracharya Institute of Professional Management and Technology, Raipur, India, in 2024.

His research interests include medical computer vision and multimodal generative AI.

Manas Wadhwa received the Bachelor of Technology degree in computer science and engineering from Indian Institute of Technology Bhilai, Bhilai, India, in 2023. He is currently pursuing the master's degree in computer science with the University of Massachusetts, Amherst, MA, USA.

His current research interests include edge machine learning and natural language processing.